

**ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ**



**BÁO CÁO BÀI TẬP MÔN
KỸ THUẬT VÀ CÔNG NGHỆ LỚN CHO TRÍ TUỆ NHÂN TẠO**

ĐỀ TÀI:
**PySpark Analysis of YouTube Video Trends,
Categories, and Interactions**

Nhóm sinh viên thực hiện: Nhóm 08
Giáo viên hướng dẫn: TS. Trần Hồng Việt
Mã học phần: AIT3229 1

Hà Nội, ngày 10 tháng 11 năm 2025

Mục lục

1	Giới thiệu Tổng quan	3
1.1	Đặt vấn đề	3
1.2	Mục tiêu Đề tài	3
1.3	Phạm vi Đề tài	4
1.4	Cấu trúc Báo cáo	4
2	Kiến trúc Hệ thống và Công nghệ sử dụng	5
2.1	Kiến trúc Tổng thể (System Architecture)	5
2.2	Chi tiết các Thành phần Hệ thống	6
2.2.1	Phân hệ Nguồn và Thu thập (Producer)	6
2.2.2	Phân hệ Xử lý Lô (Batch Layer - Offline Processing)	6
2.2.3	Phân hệ Xử lý Luồng (Speed Layer - Real-time Processing)	7
2.2.4	Hệ thống Lưu trữ (Storage)	7
2.2.5	Giao diện và API (Web Application)	8
2.3	Công nghệ và Công cụ sử dụng	8
2.3.1	Ngôn ngữ Lập trình Python	8
2.3.2	Nền tảng Apache Spark (PySpark)	8
2.3.3	Hệ thống trung chuyển thông điệp Apache Kafka	9
2.3.4	Hệ quản trị Cơ sở dữ liệu MongoDB	9
2.3.5	Backend Framework Flask	10
2.3.6	Frontend và Trực quan hóa dữ liệu	10
2.3.7	Công cụ Ảo hóa Docker & Docker Compose	10
3	Triển khai Hệ thống	11
3.1	Tổ chức Mã nguồn và Môi trường Triển khai	11
3.1.1	Cấu hình Hạ tầng (Infrastructure as Code)	11
3.1.2	Module Thu thập và Làm sạch Dữ liệu (Data Ingestion Layer)	11
3.2	Triển khai Phân tích Dữ liệu (Analytics Layer)	12
3.2.1	Phân tích Lô (Batch Analytics)	12
3.2.2	Phân tích Thời gian thực (Real-time Streaming)	12
3.3	Triển khai Tầng Ứng dụng và Trực quan hóa (Presentation Layer)	12

3.4	Quy trình Vận hành và Kiểm thử	13
4	Phân tích & Trực quan hóa Dữ liệu	14
4.1	Tiền xử lý & Làm sạch Dữ liệu	14
4.2	Phân tích Thể loại (Category)	14
4.3	Phân tích Xu hướng theo Thời gian (Notebook 04)	19
4.4	Phân tích Tương tác (Interaction) (Notebook 05)	22
5	Giám sát Thời gian thực	28
5.1	Mô hình (Sử dụng YouTube API)	28
5.2	Kết quả (Dashboard giám sát)	29
5.2.1	Hiệu suất Mô hình AI (Model Metrics)	29
5.2.2	Thống kê Tổng quan (Key Statistics)	30
5.2.3	Phân tích Biểu đồ (Visual Analytics)	30
5.2.4	Bảng Dự báo (Predictions Table)	32
6	Kết luận & Hướng phát triển	34
6.1	Kết luận	34
6.2	Khó khăn gặp phải	35
6.3	Hướng phát triển	35

Chương 1

Giới thiệu Tổng quan

1.1 Đặt vấn đề

Trong kỷ nguyên số hiện nay, YouTube đã trở thành nền tảng chia sẻ video lớn nhất toàn cầu, với hàng tỷ người dùng và hàng triệu video được tải lên mỗi ngày. Dữ liệu được tạo ra trên nền tảng này là một nguồn tài nguyên khổng lồ, phản ánh các xu hướng văn hóa, xã hội và thị trường một cách nhanh chóng.

Tuy nhiên, chính sự khổng lồ của dữ liệu này (đặc trưng bởi Volume - Khối lượng và Velocity - Tốc độ) đã tạo ra một thách thức lớn: Làm thế nào để có thể xử lý, phân tích và rút ra các thông tin chi tiết (insights) có giá trị từ luồng dữ liệu liên tục này? Việc phát hiện "thứ gì đang thịnh hành" và "tại sao nó thịnh hành" không thể được thực hiện hiệu quả bằng các công cụ phân tích truyền thống.

Nhóm quyết định lựa chọn đề tài "Phân tích và dự đoán xu hướng YouTube" nhằm xây dựng một pipeline xử lý dữ liệu hoàn chỉnh, từ việc thu thập dữ liệu thô đến việc trực quan hóa các chỉ số xu hướng (Trending) ngay lập tức.

1.2 Mục tiêu Đề tài

Mục tiêu chính của đề tài là thiết kế và triển khai một hệ thống phân tích dữ liệu lớn, bao gồm cả hai luồng xử lý hàng loạt (batch) và xử lý thời gian thực (real-time), nhằm đạt được các mục tiêu cụ thể sau:

- Xây dựng kiến trúc Lambda: Kết hợp xử lý luồng (Streaming) và xử lý lô (Batch).
- Đóng gói hệ thống (Containerization): Sử dụng Docker Compose để triển khai toàn bộ hệ sinh thái Big Data, đảm bảo tính nhất quán và dễ dàng tái lập (reproducibility).
- Phân tích: Theo dõi biến động View/Like/Comment theo thời gian thực; Phân tích theo category, thời gian, tương tác; Dự đoán views tomorrow dựa trên ML model với enhanced features (như engagement_rate, like_rate).
- Lưu trữ dữ liệu lịch sử lâu dài trên HDFS.

1.3 Phạm vi Đề tài

Đề tài tập trung vào các phạm vi cụ thể sau:

- **Nguồn dữ liệu:** YouTube Data API v3 và Kaggle dataset (US trending videos).
- **Đối tượng:** Metadata của video (Tiêu đề, thẻ tag, ngày đăng) và tương tác người dùng (View, Like, Comment).
- **Công nghệ:** Python, Apache Kafka, Apache Spark (PySpark), Hadoop HDFS, MongoDB, Flask API, HTML/JS Dashboard, Docker.

1.4 Cấu trúc Báo cáo

Nội dung báo cáo được tổ chức thành 6 chương chính:

- **Chương 1:** Giới thiệu tổng quan về đề tài, bao gồm lý do chọn đề tài, mục tiêu, phạm vi và cấu trúc của báo cáo.
- **Chương 2:** Trình bày về kiến trúc hệ thống và các công nghệ cốt lõi được sử dụng, bao gồm HDFS, Apache Spark, Docker và Flask/HTML Dashboard.
- **Chương 3:** Mô tả chi tiết các bước triển khai hệ thống, từ việc thiết lập môi trường Docker, cấu hình các dịch vụ, đến quy trình ETL (Trích xuất, Chuyển đổi, Tải) dữ liệu.
- **Chương 4:** Trình bày các kết quả phân tích và trực quan hóa dữ liệu từ luồng xử lý batch, đi sâu vào các phát hiện về Thể loại, Thời gian và Tương tác.
- **Chương 5:** Mô tả mô hình giám sát thời gian thực, cách thức hoạt động của YouTube API và kết quả hiển thị trên dashboard giám sát.
- **Chương 6:** Tổng kết lại các kết quả đạt được, nêu ra những khó khăn gặp phải trong quá trình thực hiện và đề xuất các hướng phát triển trong tương lai cho hệ thống.

Chương 2

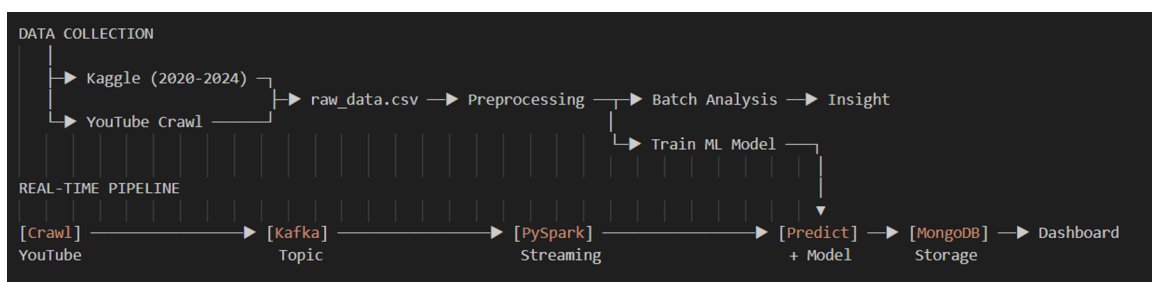
Kiến trúc Hệ thống và Công nghệ sử dụng

2.1 Kiến trúc Tổng thể (System Architecture)

Hệ thống được thiết kế dựa trên mô hình Lambda Architecture giản lược, tích hợp cả khả năng xử lý lô (Batch Processing) phục vụ huấn luyện mô hình và xử lý luồng (Stream Processing) phục vụ phân tích thời gian thực.

Kiến trúc bao gồm 4 tầng chính:

1. **Tầng Thu thập (Data Ingestion Layer):** Chịu trách nhiệm kết nối với nguồn dữ liệu bên ngoài (YouTube API) và đẩy dữ liệu vào đường ống xử lý.
2. **Tầng Xử lý & Phân tích (Processing & Analytics Layer):** Trái tim của hệ thống, nơi diễn ra các tác vụ làm sạch, tính toán thống kê và dự báo bằng Machine Learning.
3. **Tầng Lưu trữ (Storage Layer):** Quản lý việc lưu trữ dữ liệu thô, dữ liệu đã qua xử lý và kết quả dự báo.
4. **Tầng Trình diễn (Presentation Layer):** Cung cấp giao diện trực quan cho người dùng cuối và API để truy xuất dữ liệu.



Hình 2.1: Sơ đồ kiến trúc tổng thể hệ thống

Mô hình hoạt động theo cơ chế Event-Driven (hướng sự kiện): Dữ liệu video mới sinh ra sẽ kích hoạt chuỗi hành động xử lý tức thì từ Kafka đến Spark và cuối cùng là Dashboard.

2.2 Chi tiết các Thành phần Hệ thống

2.2.1 Phân hệ Nguồn và Thu thập (Producer)

Nguồn dữ liệu:

- **Offline:** Dataset lịch sử từ Kaggle và dữ liệu thu thập định kỳ từ YouTube Data API v3 để xây dựng kho dữ liệu huấn luyện.
- **Real-time:** Dữ liệu video trending được truy xuất liên tục từ YouTube API v3 (metadata, view, like, comment).

Kafka Producer:

Được xây dựng bằng Python (`kafka-python`), module này hoạt động như một dịch vụ nền (background service). Cơ chế hoạt động: Định kỳ gửi yêu cầu (request) tới YouTube API, nhận kết quả JSON, tuần tự hóa (serialize) và đẩy vào Kafka Topic có tên `youtube-trending`.

2.2.2 Phân hệ Xử lý Lô (Batch Layer - Offline Processing)

Đây là phân hệ hoạt động "ngoại tuyến"(offline), đóng vai trò nền tảng cho trí tuệ của hệ thống. Phân hệ này không yêu cầu phản hồi tức thì mà tập trung vào việc xử lý khối lượng dữ liệu lớn đã được lưu trữ.

- **Đầu vào:** Dữ liệu lịch sử dạng CSV (`preprocessed_data`) bao gồm hàng nghìn video trending trong quá khứ.
- **Các tác vụ chính:**
 1. **Tiền xử lý & Làm sạch (Cleaning):** Xử lý giá trị thiếu (null), chuẩn hóa định dạng thời gian, và loại bỏ nhiễu.
 2. **Trích xuất đặc trưng (Feature Engineering):** Tính toán các chỉ số nâng cao để phục vụ học máy, bao gồm:
 - *Engagement Rate*: Tỷ lệ tương tác tổng hợp.
 - *Like Rate*: Tỷ lệ thích trên lượt xem.
 - *View Scale (Log10)*: Chuẩn hóa lượt xem về thang đo logarit.
 - *Prime Time*: Xác định khung giờ vàng đăng tải.

3. **Phân tích Khám phá (EDA):** Chạy các notebook phân tích sâu để tìm ra các "luật"(rules) và "ngưỡng"(thresholds) của video thịnh hành (ví dụ: ngưỡng viral top 5%, giờ đăng hiệu quả nhất).
 4. **Huấn luyện Mô hình (Model Training):** Sử dụng thư viện Spark MLlib để huấn luyện thuật toán Gradient Boosted Trees (GBTRegressor). Mô hình này học từ dữ liệu lịch sử để hiểu mối quan hệ giữa các đặc trưng và sự tăng trưởng lượt xem.
- **Đầu ra:** Một mô hình đã được huấn luyện (Pre-trained Model) và các thông số baseline (đường cơ sở) được lưu lại để cung cấp cho luồng xử lý thời gian thực.

2.2.3 Phân hệ Xử lý Luồng (Speed Layer - Real-time Processing)

Đây là thành phần phức tạp nhất, được triển khai trong file `07_realtime_analysis.py`, kết hợp giữa Spark SQL và Spark MLlib.

Huấn luyện Mô hình (Batch Phase):

- Trước khi bắt đầu xử lý luồng, hệ thống khởi động Spark Session để đọc dữ liệu lịch sử từ CSV (`preprocessed_data`).
- Thực hiện trích xuất đặc trưng (Feature Engineering) nâng cao như: Engagement Rate, Like Rate, View Scale (Log10), Prime Time (Khung giờ vàng).
- Sử dụng thuật toán Gradient Boosted Trees (GBTRegressor) để huấn luyện mô hình dự báo tiềm năng tăng trưởng (`growth_potential`).

Xử lý Luồng (Streaming Phase):

- Đóng vai trò là Kafka Consumer, lắng nghe liên tục topic `youtube-trending`.
- Ngay khi nhận gói tin, hệ thống áp dụng mô hình đã huấn luyện (pre-trained model) để dự báo lượt xem ngày hôm sau (`predicted_views_tomorrow`) cho video đó.
- Tính toán các chỉ số thống kê tổng hợp theo danh mục (Category Stats) trong thời gian thực.

2.2.4 Hệ thống Lưu trữ (Storage)

MongoDB: Được lựa chọn làm cơ sở dữ liệu chính nhờ khả năng xử lý JSON linh hoạt (Schema-less), phù hợp với cấu trúc dữ liệu thay đổi của mạng xã hội.

Tổ chức dữ liệu: Dữ liệu sau khi xử lý từ Spark được phân tách và lưu vào 3 collections riêng biệt để tối ưu hiệu năng truy vấn:

- `realtime_data`: Lưu trữ thông tin chi tiết từng video.

- `category_stats`: Lưu trữ bảng thống kê tổng hợp.
- `predictions`: Lưu trữ danh sách video có tiềm năng tăng trưởng cao nhất.

2.2.5 Giao diện và API (Web Application)

- **Backend API (Flask)**: Cung cấp các RESTful endpoints (`/api/realtime`, `/api/predictions`) cho phép truy xuất dữ liệu từ MongoDB một cách an toàn và bảo mật thông qua cơ chế CORS.
- **Frontend Dashboard**: Sử dụng JavaScript để thực hiện cơ chế Long Polling (gọi API mỗi 5 giây), cập nhật các biểu đồ Chart.js mà không cần tải lại trang. Dashboard cũng tích hợp logic phía client để tính toán "Điểm tin cậy" (Confidence Score) cho các dự báo của AI.

2.3 Công nghệ và Công cụ sử dụng

Hệ thống được xây dựng dựa trên hệ sinh thái mã nguồn mở (Open Source) mạnh mẽ, kết hợp giữa ngôn ngữ lập trình linh hoạt Python và các nền tảng Big Data chuyên dụng.

2.3.1 Ngôn ngữ Lập trình Python

Python (phiên bản 3.x) được lựa chọn làm ngôn ngữ chủ đạo xuyên suốt toàn bộ dự án, từ khâu thu thập dữ liệu (ETL) đến xây dựng Backend. Vai trò: Là ngôn ngữ "keo dính" (glue language) kết nối các thành phần hệ thống.

Các thư viện cốt lõi:

- `google-api-python-client`: Thư viện chính chủ của Google, được sử dụng trong module Producer để xác thực và gửi yêu cầu truy vấn đến YouTube Data API v3. Nó hỗ trợ phân trang (pagination) và lọc các trường dữ liệu cần thiết.
- `kafka-python`: Client thực hiện giao thức Kafka cho Python, chịu trách nhiệm tuần tự hóa (serialization) dữ liệu dictionary sang định dạng JSON byte-stream trước khi đẩy vào Message Queue.
- `pandas` & `numpy`: Sử dụng trong các Notebook phân tích để thao tác với cấu trúc dữ liệu DataFrame, thực hiện tính toán số học ma trận và xử lý các giá trị Null/NaN trong giai đoạn tiền xử lý.

2.3.2 Nền tảng Apache Spark (PySpark)

Apache Spark đóng vai trò là "trái tim" xử lý của hệ thống, cung cấp khả năng tính toán phân tán tốc độ cao. Dự án sử dụng API PySpark để tương tác với Spark Core viết bằng Scala.

- **Spark Structured Streaming:** Thay vì xử lý theo lô (batch) truyền thống với độ trễ cao, hệ thống sử dụng Structured Streaming để xử lý dữ liệu thời gian thực. Spark Session được cấu hình với gói `spark-sql-kafka` để đọc trực tiếp luồng dữ liệu từ Kafka Topic như một bảng vô hạn (unbounded table).
- **Spark MLlib (Machine Learning Library):** Module `pyspark.ml` được sử dụng để xây dựng pipeline học máy.
 - *VectorAssembler:* Công cụ dùng để gộp các cột đặc trưng rời rạc (như `likes`, `view_count`, `categoryId`) thành một vector đặc trưng duy nhất làm đầu vào cho mô hình.
 - *GBTRegressor (Gradient-Boosted Trees):* Thuật toán hồi quy dựa trên cây quyết định được lựa chọn thay vì Linear Regression hay Random Forest. GBT cho độ chính xác cao hơn trong việc dự báo các biến số liên tục (như lượng view) nhờ cơ chế tối ưu hóa sai số qua từng cây yếu (weak learners).

2.3.3 Hệ thống trung chuyển thông điệp Apache Kafka

Trong kiến trúc hướng sự kiện (Event-driven Architecture) của dự án, Apache Kafka đóng vai trò là lớp đệm (buffer) trung gian quan trọng.

- **Cơ chế Pub/Sub:** Kafka hoạt động theo cơ chế Publish-Subscribe. `kafka_producer.py` đóng vai trò là Publisher đẩy dữ liệu vào Topic `youtube-trending`, trong khi Spark Job đóng vai trò là Subscriber tiêu thụ dữ liệu đó.
- **Khả năng chịu lỗi (Fault Tolerance):** Kafka giúp hệ thống không bị mất dữ liệu ngay cả khi quá trình xử lý (Spark) bị gián đoạn. Dữ liệu được lưu trữ bền vững trong topic và có thể được replay (phát lại) khi cần thiết.
- **Cấu hình:** Producer được thiết lập với `acks='all'` và `retries=3` để đảm bảo tính toàn vẹn dữ liệu cao nhất.

2.3.4 Hệ quản trị Cơ sở dữ liệu MongoDB

MongoDB là cơ sở dữ liệu NoSQL dạng Document, được lựa chọn thay thế cho RDBMS (như MySQL) nhờ tính linh hoạt của Schema.

- **Lưu trữ JSON tự nhiên:** Dữ liệu từ YouTube API có cấu trúc phân cấp phức tạp (nested JSON). MongoDB cho phép lưu trữ nguyên trạng các object này mà không cần thực hiện kỹ thuật ORM phức tạp.
- **Phân loại dữ liệu lưu trữ:** Hệ thống chia tách dữ liệu thành 3 bộ sưu tập (collections) chuyên biệt trong database `youtube_analytics`: `realtime_data`, `category_stats`, `predictions`.

2.3.5 Backend Framework Flask

Để kết nối lớp dữ liệu (Data Layer) và lớp hiển thị (Presentation Layer), Flask được sử dụng để xây dựng RESTful API Server nhẹ (Micro-service).

- **Định tuyến (Routing):** Flask cung cấp các URL endpoints cụ thể (ví dụ: `/api/realtime`) để Frontend có thể gửi yêu cầu HTTP GET lấy dữ liệu.
- **CORS:** Module `flask_cors` được tích hợp để giải quyết vấn đề bảo mật trình duyệt.
- **Tích hợp PyMongo:** Flask sử dụng driver `pymongo` để thực hiện các truy vấn đọc dữ liệu tốc độ cao.

2.3.6 Frontend và Trực quan hóa dữ liệu

Giao diện người dùng (Dashboard) được xây dựng theo hướng Single Page Application (SPA) đơn giản, tập trung vào trải nghiệm thời gian thực.

- **Chart.js:** Thư viện JavaScript mã nguồn mở sử dụng HTML5 Canvas để vẽ biểu đồ. Chart.js được chọn vì dung lượng nhẹ, hỗ trợ animation mượt mà. Hệ thống sử dụng đa dạng các loại biểu đồ: Bar Chart, Line Chart và Grouped Bar.
- **Cơ chế Long Polling:** Thay vì sử dụng WebSockets phức tạp, Dashboard sử dụng hàm `setInterval` của JavaScript kết hợp với fetch API để định kỳ 5 giây/lần gửi yêu cầu cập nhật dữ liệu từ Server, tạo cảm giác "Real-time".

2.3.7 Công cụ Ảo hóa Docker & Docker Compose

Để giải quyết vấn đề "It works on my machine", toàn bộ hạ tầng được đóng gói bằng Docker.

- **Containerization:** Các dịch vụ nền tảng như Zookeeper, Kafka broker và MongoDB server được chạy dưới dạng các Container độc lập.
- **Orchestration:** File `docker-compose.yml` định nghĩa toàn bộ stack công nghệ, mạng ảo (virtual network) và các volume lưu trữ, cho phép khởi tạo toàn bộ hệ thống phân tán phức tạp chỉ bằng một câu lệnh `docker-compose up`.

Chương 3

Triển khai Hệ thống

3.1 Tổ chức Mã nguồn và Môi trường Triển khai

3.1.1 Cấu hình Hạ tầng (Infrastructure as Code)

Hệ thống không cài đặt thủ công mà được tự động hóa qua Docker:

- File `docker-compose.yml`: Định nghĩa các dịch vụ (services) cần thiết bao gồm:
 - `zookeeper`: Quản lý cluster cho Kafka.
 - `kafka`: Broker xử lý luồng tin nhắn.
 - `mongodb`: Nơi lưu trữ kết quả cuối cùng (Sink).
- File `dockerfile`: Dùng để build image chứa môi trường Python, cài sẵn các thư viện cần thiết (`pymongo`, `kafka-python`, `pandas`,...) giúp đảm bảo code chạy ổn định trên mọi máy.

3.1.2 Module Thu thập và Làm sạch Dữ liệu (Data Ingestion Layer)

Giai đoạn này tập trung vào việc đưa dữ liệu thô vào hệ thống (ETL - Extract & Transform):

- **Thu thập dữ liệu lịch sử** (`00_download_kaggle_data.ipynb`): Module này tải các tập dữ liệu lớn có sẵn từ Kaggle để phục vụ việc huấn luyện mô hình hoặc phân tích xu hướng dài hạn mà API hiện tại không cung cấp đủ.
- **Thu thập dữ liệu thực tế** (`01_download_youtube_data.ipynb`): Sử dụng `google-api-python-client` để truy vấn YouTube Data API v3. Dữ liệu thu về bao gồm: Thông tin Video (Title, Tags, Duration), Chỉ số tương tác (Views, Likes, Comments) và Thông tin Kênh.
- **Tiền xử lý dữ liệu** (`02_preprocessing.ipynb`): Thực hiện làm sạch dữ liệu (Data Cleaning): Xử lý các giá trị bị thiếu (Null/NaN), loại bỏ các ký tự đặc biệt trong văn bản comment. Chuẩn hóa dữ liệu (Normalization): Chuyển đổi định dạng ngày tháng (ISO 8601) về dạng datetime chuẩn, chuyển đổi kiểu dữ liệu số.

3.2 Triển khai Phân tích Dữ liệu (Analytics Layer)

3.2.1 Phân tích Lô (Batch Analytics)

Các Notebooks được sử dụng để trả lời các câu hỏi nghiệp vụ sâu sắc dựa trên dữ liệu đã lưu trữ:

- **Phân tích Danh mục** (03_category_analysis.ipynb): Thống kê số lượng video theo từng Category ID. Xác định danh mục nào thu hút nhiều lượt xem nhất (Gaming, Music, Education,...).
- **Phân tích Chuỗi thời gian** (04_time_analysis.ipynb): Phân tích tần suất đăng video theo giờ trong ngày và ngày trong tuần. Biểu diễn xu hướng tăng trưởng lượt xem theo thời gian thực của video.
- **Phân tích Tương tác** (05_interation_analysis.ipynb): Tính toán chỉ số Engagement Rate (Tỷ lệ tương tác). Phân tích mối tương quan (Correlation) giữa độ dài video và lượt xem, hoặc giữa số lượng tags và thứ hạng tìm kiếm.

3.2.2 Phân tích Thời gian thực (Real-time Streaming)

Đây là thành phần phức tạp nhất, xử lý dữ liệu ngay khi nó sinh ra:

- **Kafka Producer** (06_kafka_producer.py): Hoạt động như một nguồn phát tin (Source). Script này liên tục lấy dữ liệu mới (hoặc giả lập dữ liệu stream) và gửi (push) vào Kafka Topic có tên youtube_stream. Dữ liệu được serialize dưới dạng JSON trước khi gửi.
- **Kafka Consumer & Processing** (07_realtime_analysis.py): Đóng vai trò Consumer lắng nghe Topic youtube_stream. Ngay khi nhận được gói tin, hệ thống thực hiện phân tích tức thì (ví dụ: đếm số lượng comment tiêu cực/tích cực trong 1 phút gần nhất). Kết quả sau xử lý được ghi trực tiếp vào MongoDB để phục vụ hiển thị.

3.3 Triển khai Tầng Ứng dụng và Trực quan hóa (Presentation Layer)

Để người dùng cuối có thể khai thác thông tin, hệ thống cung cấp giao diện trực quan:

- **Backend API** (09_api_server.py): Sử dụng framework nhẹ (như Flask/FastAPI) để xây dựng RESTful API. Cung cấp các endpoints (ví dụ: /api/get_stats, /api/realtime_data) cho phép Frontend truy xuất dữ liệu từ MongoDB mà không cần kết nối trực tiếp vào Database.
- **Frontend Dashboard** (08_dashboard.html): Sử dụng HTML/JS kết hợp với thư viện biểu đồ (như Chart.js hoặc Highcharts). Cơ chế AJAX/Fetch API được dùng để cập nhật biểu đồ tự động (auto-refresh) mỗi khi có dữ liệu mới từ API, tạo cảm giác thời gian thực (Real-time Dashboard).

3.4 Quy trình Vận hành và Kiểm thử

(Mô tả dựa trên file `Cach_chay.txt`) Để khởi chạy toàn bộ hệ thống, quy trình thực hiện như sau:

- **Bước 1:** Khởi động hạ tầng bằng lệnh `docker-compose up -d`. Kiểm tra trạng thái các container qua Docker Desktop.
- **Bước 2:** (Tùy chọn) Chạy script `clear_mongodb.py` để làm sạch database cũ, đảm bảo môi trường sạch.
- **Bước 3:** Kích hoạt luồng xử lý:
 - Terminal 1: Chạy Producer `python 06_kafka_producer.py`.
 - Terminal 2: Chạy Consumer `python 07_realtime_analysis.py`.
- **Bước 4:** Khởi động Web Server `python 09_api_server.py`.
- **Bước 5:** Truy cập Dashboard qua trình duyệt, quan sát các biểu đồ biến động.

Chương 4

Phân tích & Trực quan hóa Dữ liệu

Chương này trình bày chi tiết về quá trình phân tích và trực quan hóa dữ liệu lịch sử, được thực hiện thông qua các sổ tay PySpark. Các kết quả này không chỉ cung cấp cái nhìn sâu sắc về các xu hướng đã qua mà còn tạo ra các "baseline"(đường cơ sở) cho việc giám sát thời gian thực.

4.1 Tiền xử lý & Làm sạch Dữ liệu

Quy trình bắt đầu bằng việc đọc dữ liệu CSV thô. Sau đó, chúng tôi thực hiện các bước làm sạch bao gồm:

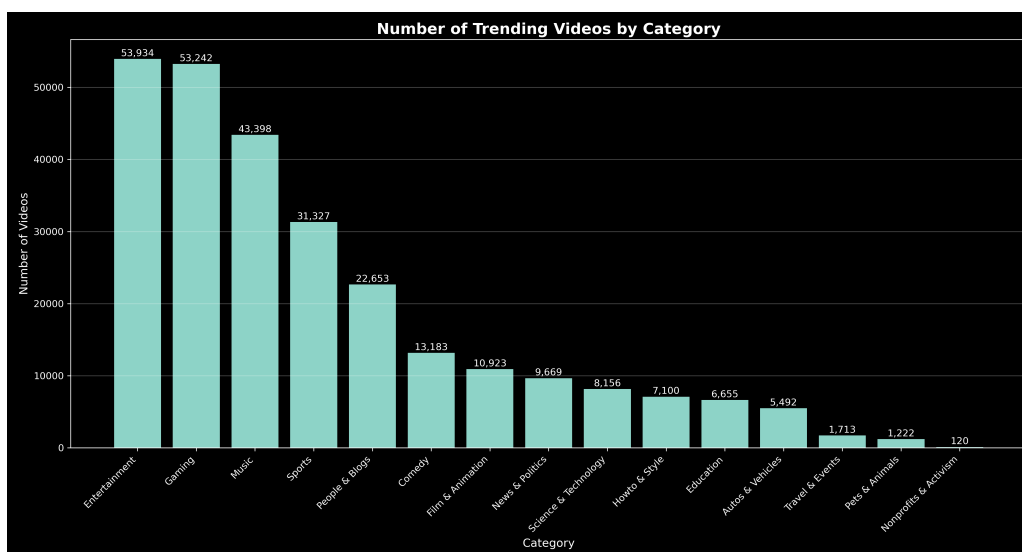
- Loại bỏ các cột không cần thiết cho phân tích như `thumbnail_link`, `comments_disabled`, và `ratings_disabled`.
- Xử lý giá trị null, đặc biệt là điền giá trị "No description" cho các video bị thiếu mô tả.
- Chuẩn hóa các cột ngày tháng (`publishedAt` và `trending_date`) sang định dạng Timestamp chuẩn để phục vụ cho các phân tích chuỗi thời gian.

Bước quan trọng nhất trong notebook này là làm giàu dữ liệu (enrichment). Chúng tôi đọc tệp `category_id.json`, xử lý nó thành một DataFrame tra cứu (lookup table), và sau đó thực hiện phép nối (join) với DataFrame video chính. Thao tác này đã thêm cột `category_name` (ví dụ: "Music", "Gaming") vào từng dòng dữ liệu, một thông tin then chốt cho các phân tích sau này. Kết quả cuối cùng là một DataFrame sạch, đã được làm giàu, được lưu lại dưới định dạng CSV để tối ưu hóa hiệu suất đọc/ghi cho các notebook phân tích tiếp theo và cho ML (`ml_data`).

4.2 Phân tích Thể loại (Category)

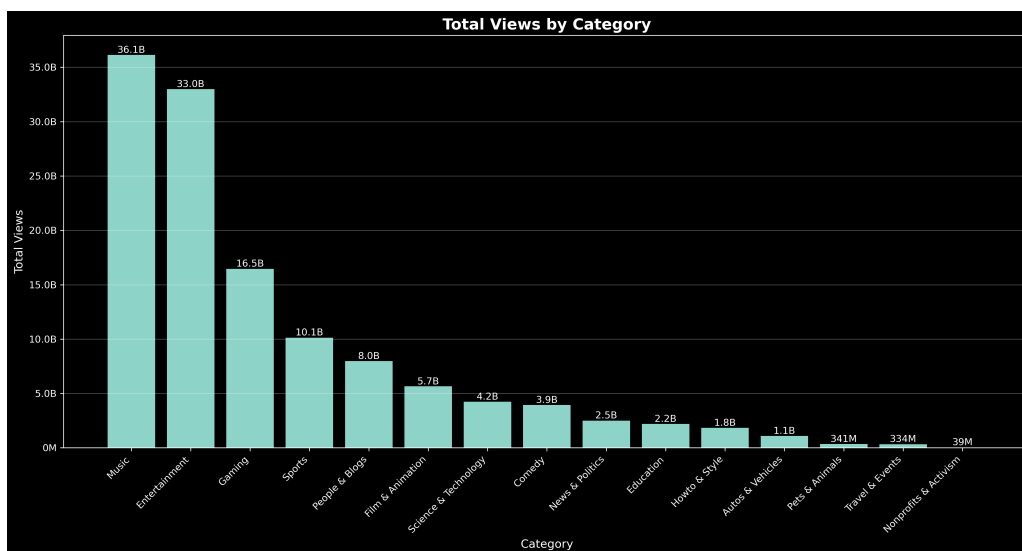
Sổ tay `03_category_analysis.ipynb` tập trung vào việc khám phá các đặc tính của từng thể loại video.

- **Tần suất xuất hiện:** Phân tích đầu tiên cho thấy Entertainment (Giải trí) và Gaming (Trò chơi) là hai thể loại xuất hiện trên top trending thường xuyên nhất, theo sau là Music (Âm nhạc). Ba thể loại này chiếm phần lớn các video thịnh hành.



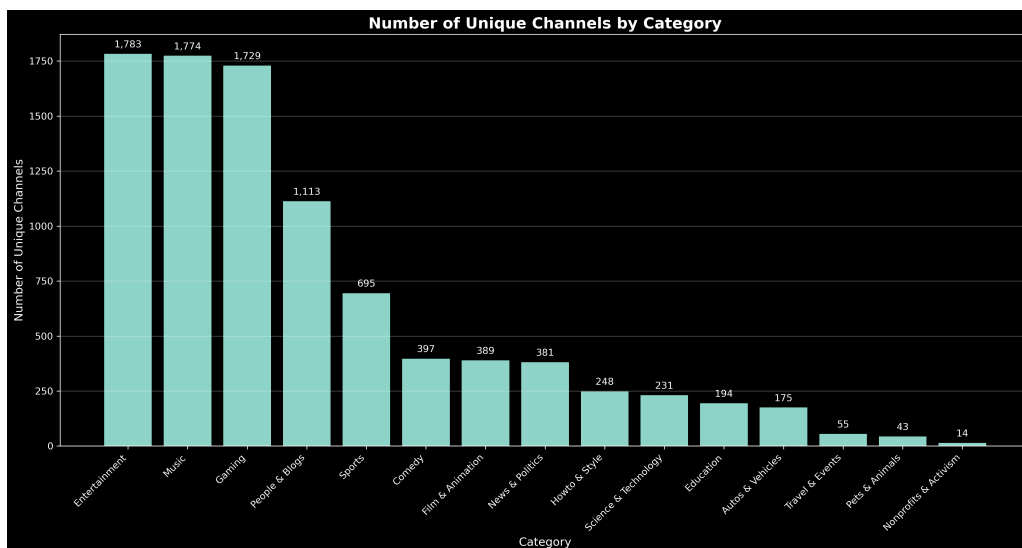
Hình 4.1: Tần suất xuất hiện của các thể loại trên Top Trending

- **Tổng lượt xem:** Mặc dù chỉ đứng thứ 3 về tần suất, Music lại dẫn đầu tuyệt đối về tổng lượt xem (36.1 tỷ), cho thấy các MV có sức hút khổng lồ. Entertainment theo sát với 33.0 tỷ lượt xem.



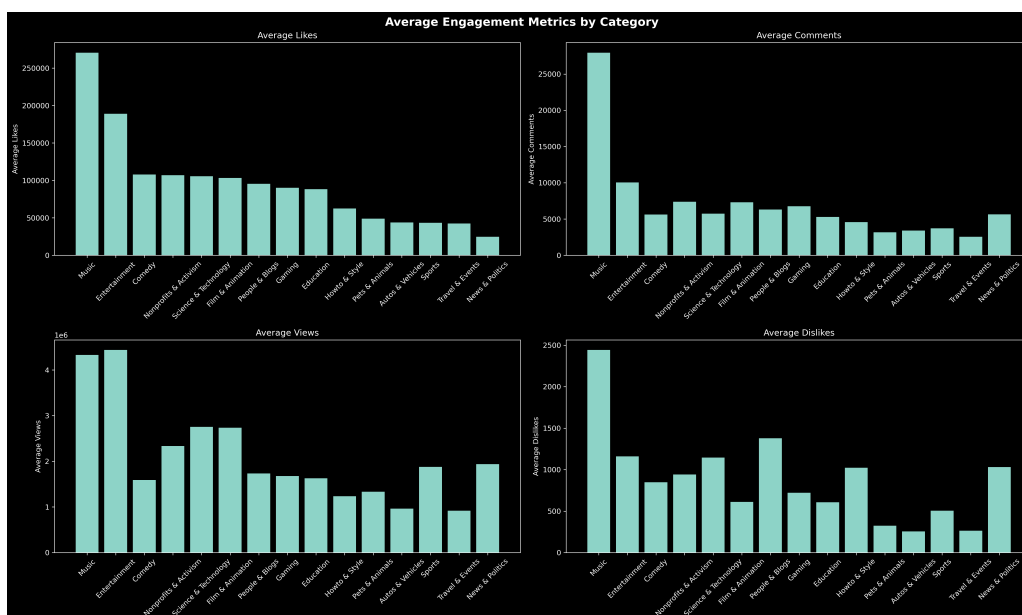
Hình 4.2: Tổng lượt xem theo từng thể loại

- **Độ đa dạng kênh:** Entertainment, Music, và Gaming cũng là ba thể loại có số lượng kênh duy nhất tham gia trending nhiều nhất (trên 1,700 kênh mỗi loại), cho thấy đây là những "sân chơi" có tính cạnh tranh cao.

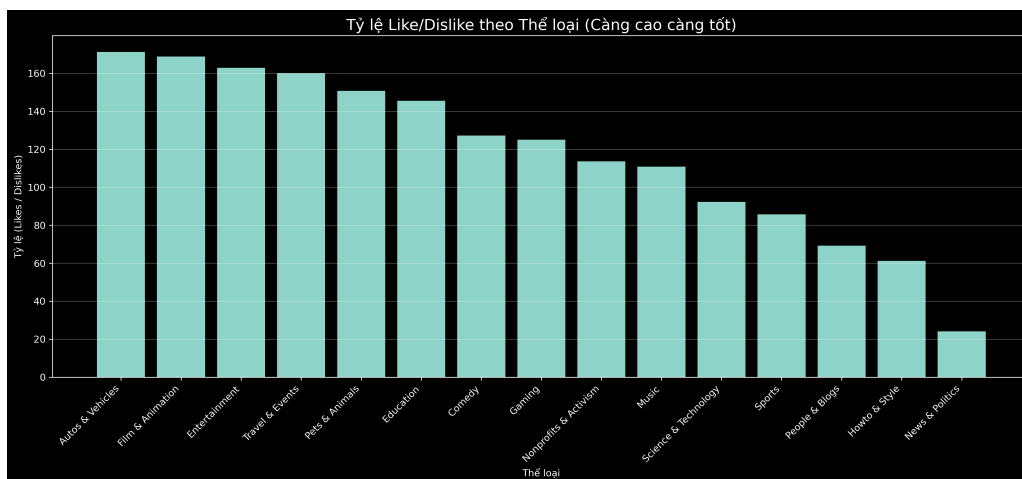


Hình 4.3: Độ đa dạng kênh trong các thể loại

- **Mức độ tương tác:** Music tiếp tục dẫn đầu về lượt Like và Comment trung bình trên mỗi video. Ngược lại, News & Politics có tỷ lệ Like/Dislike thấp nhất, cho thấy tính chất gây tranh cãi, trong khi Autos & Vehicles là thể loại “lành” nhất (tỷ lệ L/D cao nhất).

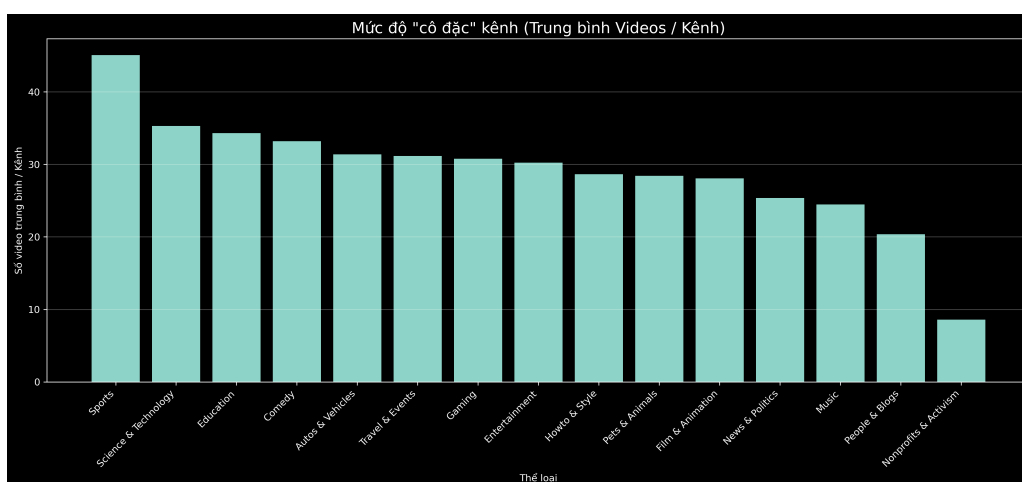


Hình 4.4: Mức độ tương tác trung bình theo thể loại



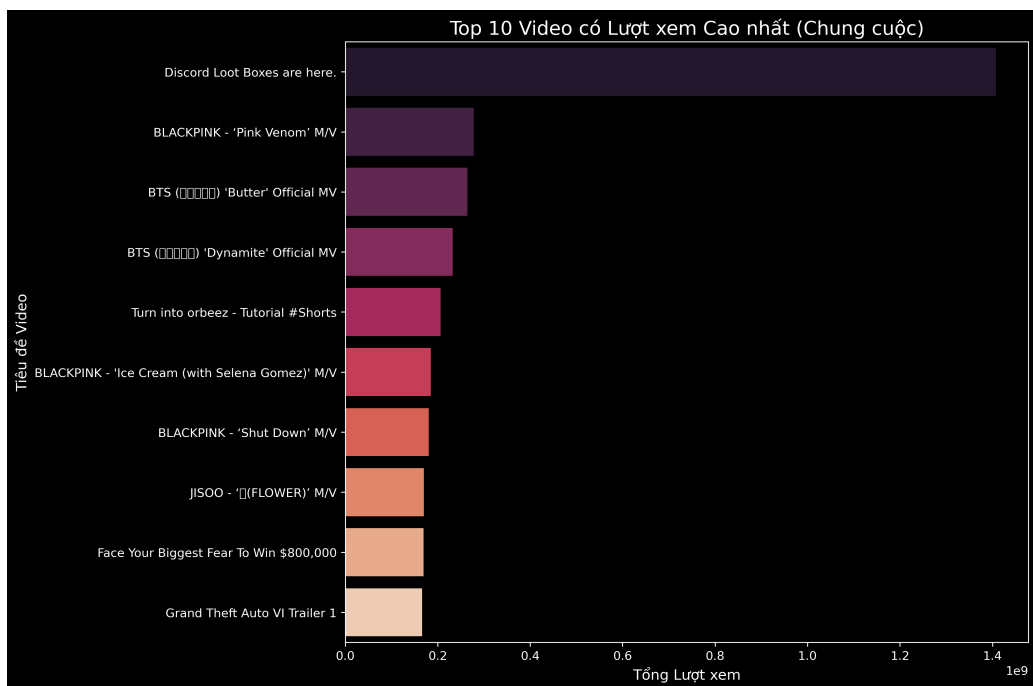
Hình 4.5: Tỷ lệ like/dislike theo thể loại

- **Mức độ “cô đặc” kênh:** Một phát hiện thú vị là thể loại Sports có chỉ số video/kênh cao nhất (45). Điều này ngụ ý rằng một số ít các kênh thể thao lớn (như ESPN, NBA) tạo ra phần lớn các video trending.

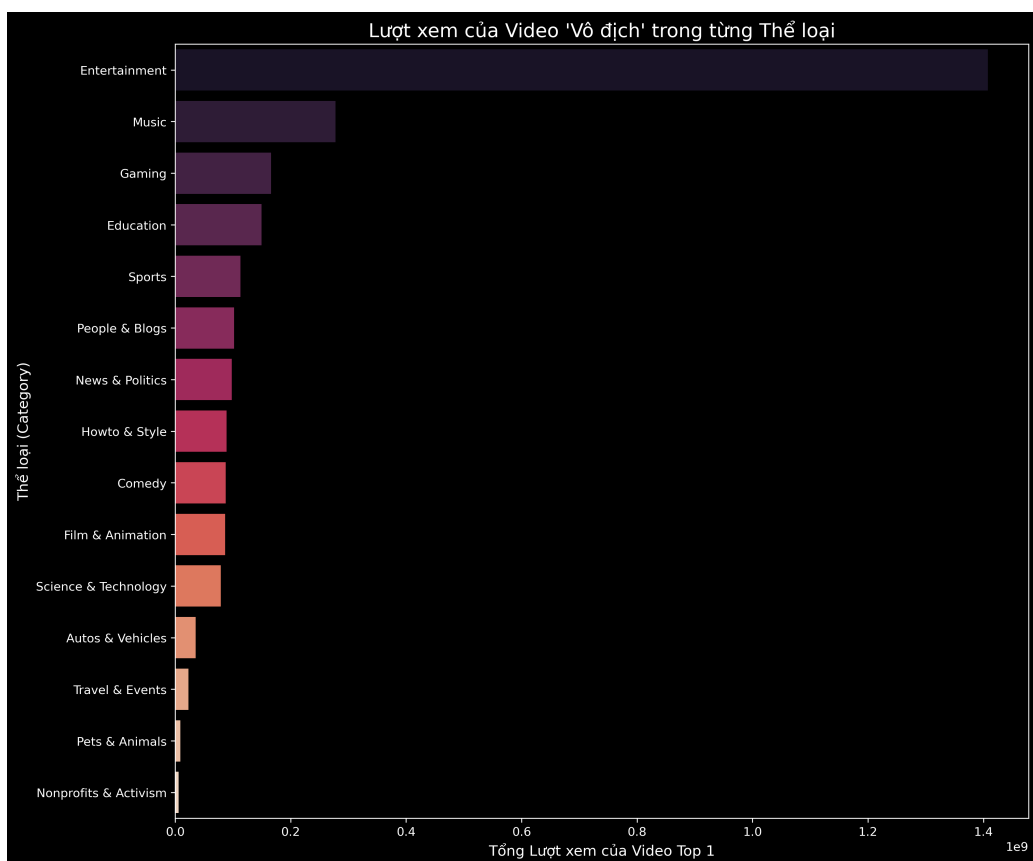


Hình 4.6: Mức độ "cô đặc" kênh

- **Video hàng đầu:** Video “Discord Loot Boxes” (Entertainment) thống trị tuyệt đối với 1.4 tỷ lượt xem. Phần còn lại của Top 10 bị chiếm lĩnh bởi K-Pop (BTS, BLACKPINK).



Hình 4.7: Các video có lượt xem cao nhất



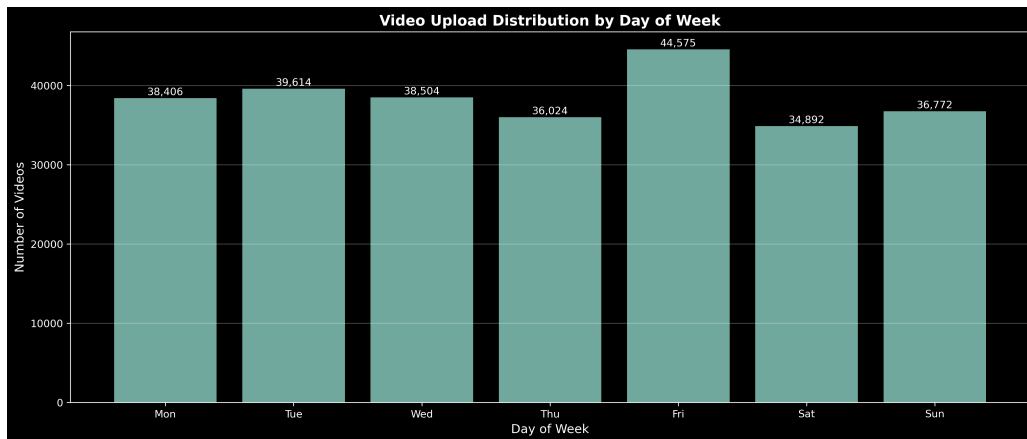
Hình 4.8: Lượt xem của Video "vô địch" từng thể loại

Kết luận từ Chương 03: Baseline về "thị phần" của các thể loại đã được thiết lập. Music và Entertainment là hai thể lực chính về lượt xem, trong khi News là thể loại gây tranh cãi nhất.

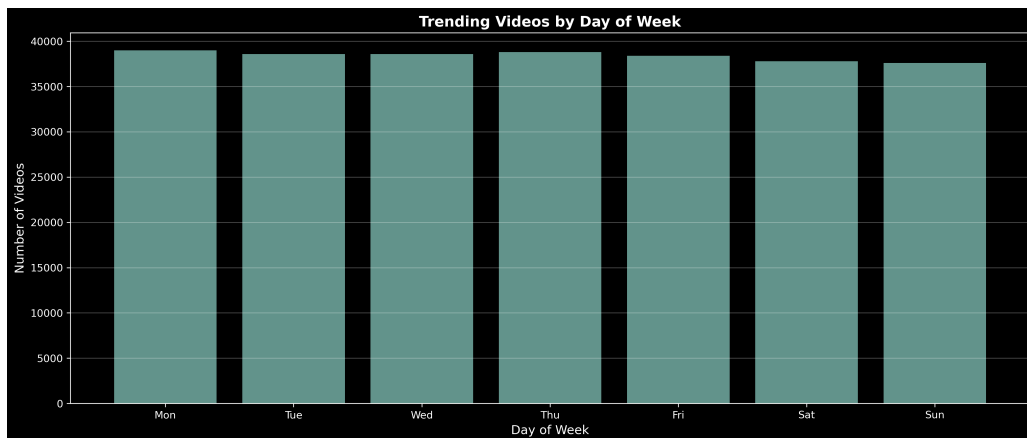
4.3 Phân tích Xu hướng theo Thời gian (Notebook 04)

Sổ tay 04_time_analysis.ipynb đi sâu vào việc "khi nào" và "trong bao lâu" một video trở nên thịnh hành.

- **Xu hướng đăng bài theo Ngày:** Thứ Sáu (Fri) là ngày "bùng nổ" nhất với hơn 44,500 video được đăng (để đón đầu cuối tuần). Tuy nhiên, biểu đồ "Video Trending" lại cho thấy YouTube phân bổ số lượng video thịnh hành rất đều cho mọi ngày trong tuần.

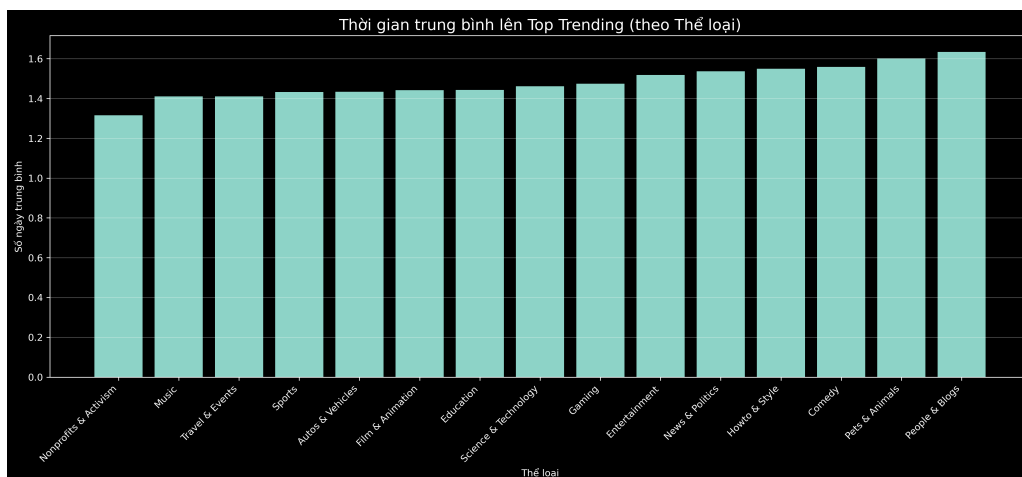


Hình 4.9: Số lượng video đăng tải theo các ngày trong tuần

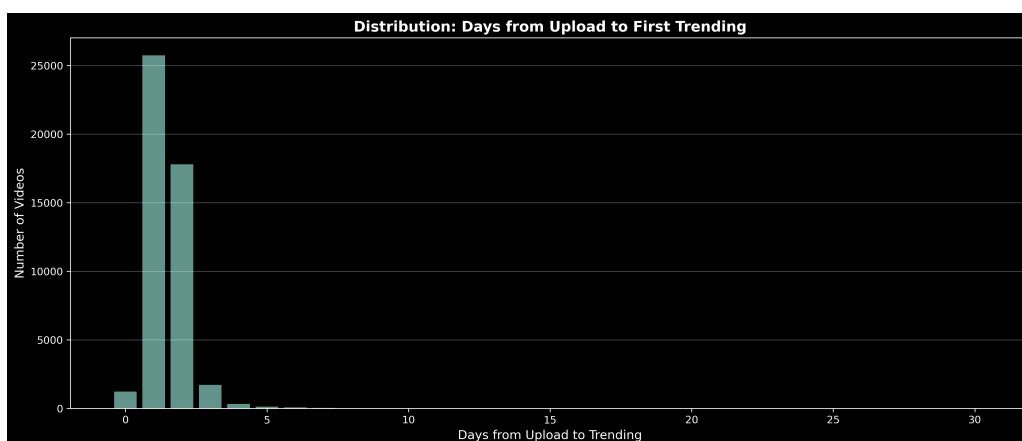


Hình 4.10: Số lượng video Trending theo các ngày trong tuần

- **Tốc độ lọt Top Trending:** Phân tích cho thấy tốc độ là yếu tố then chốt. Phần lớn các video (hơn 25,000) lọt vào trending chỉ 1 ngày sau khi đăng. Gần như không thể lọt top trending lần đầu nếu video đã quá 4-5 ngày tuổi.

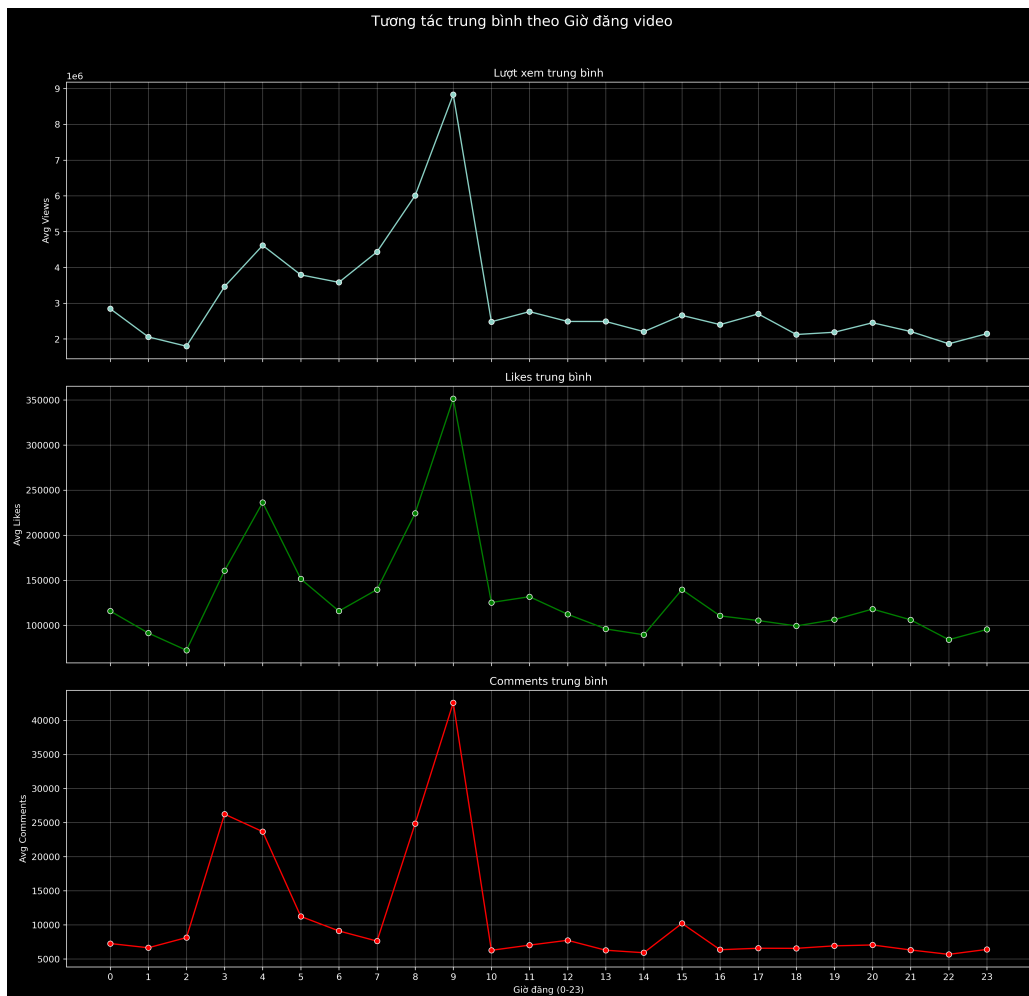


Hình 4.11: Thời gian từ khi đăng đến khi lọt Top Trending



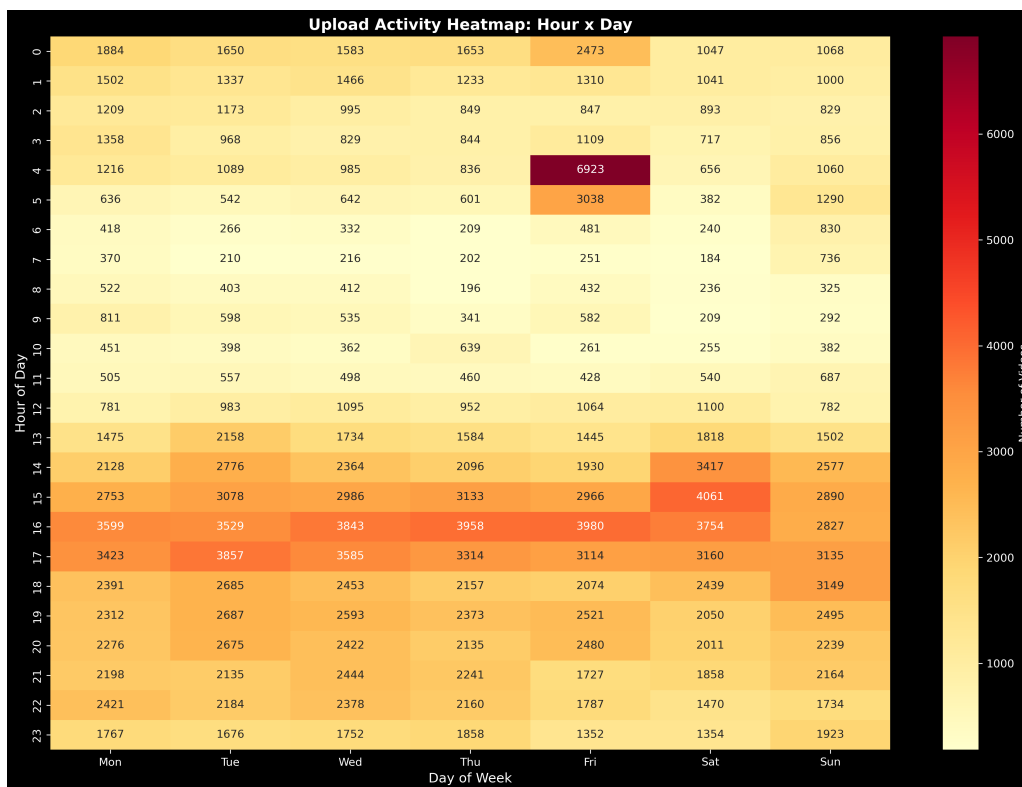
Hình 4.12: Thời gian từ khi đăng đến khi lọt Top Trending

- **Phân tích sâu:** Phân tích bổ sung đưa ra một insight quan trọng. Mặc dù thời điểm có lượng video được đăng nhiều nhất là 05:00 sáng Thứ Sáu, các video đăng lúc 09:00 sáng mới là nhóm có lượt xem, like và comment trung bình cao nhất. Điều này cho thấy việc đăng video sớm hơn (ít cạnh tranh hơn) giúp video có cả ngày để tích lũy tương tác.



Hình 4.13: Tương tác trung bình theo giờ đăng video

– **Heatmap Giờ-Ngày:** Biểu đồ nhiệt xác định rõ "giờ vàng" để đăng video.



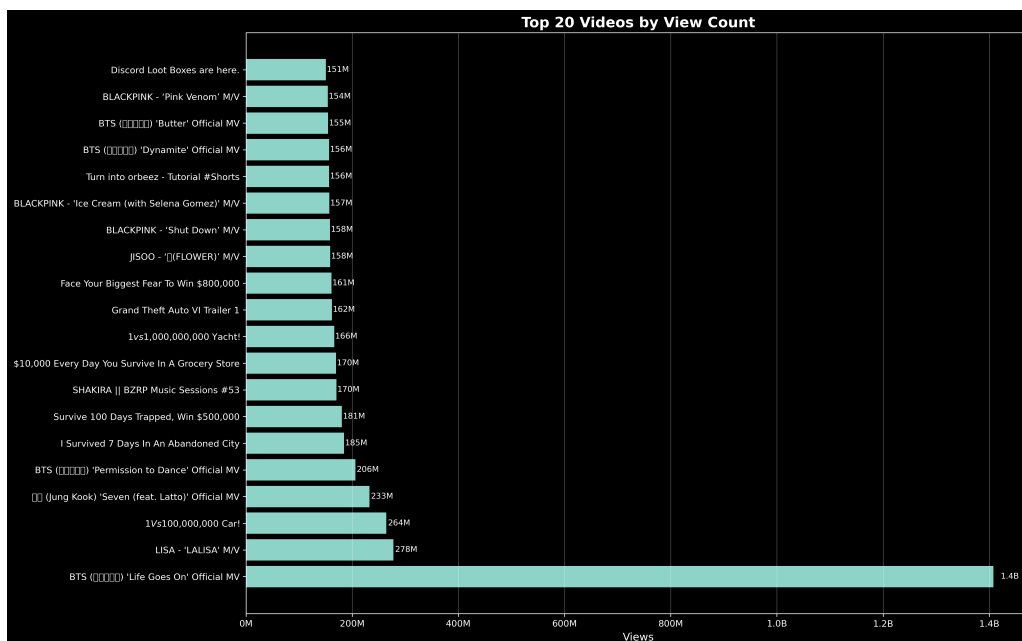
Hình 4.14: Biểu đồ nhiệt thời gian đăng bài hiệu quả

Kết luận từ Chương 04: Baseline về thời gian đã được thiết lập. Chúng tôi xác định thời điểm có lượng video được đăng nhiều nhất là 05:00 sáng Thứ Sáu. Tuy nhiên, khung giờ 15:00–18:00 (đặc biệt 17:00 Thứ Sáu) vẫn là khoảng thời gian đăng tải phổ biến và ổn định., nhưng 09:00 sáng là giờ đăng hiệu quả nhất. Đa số video phải lọt top trong vòng 48 giờ.

4.4 Phân tích Tương tác (Interaction) (Notebook 05)

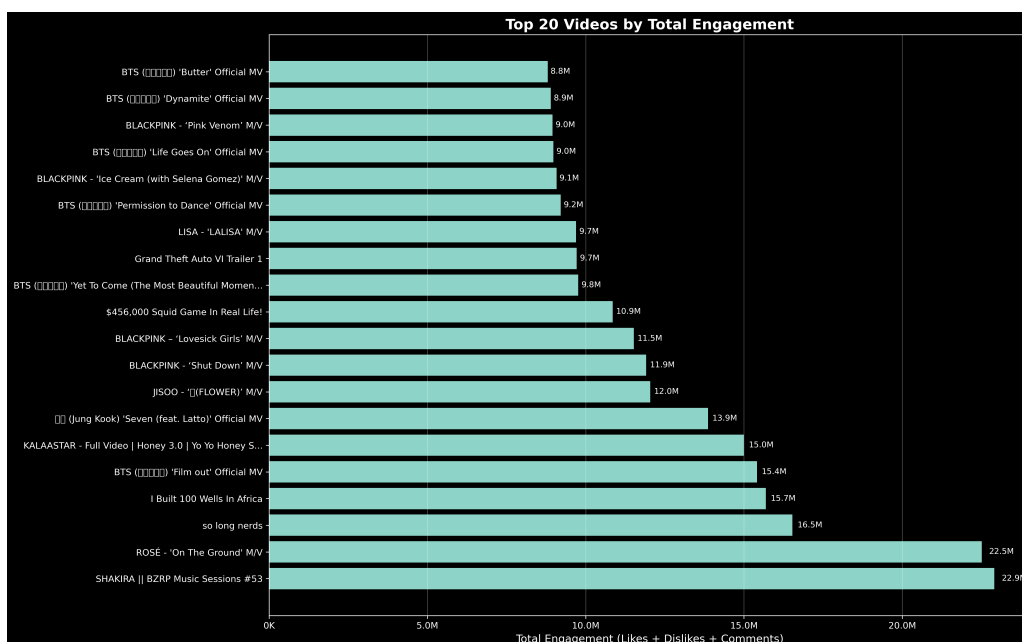
Sổ tay 05_interaction_analysis.ipynb khám phá mối quan hệ giữa các chỉ số tương tác, xác định "vua tương tác" và "chất lượng" của lượt xem.

- **Top 20 Videos:** Các bảng xếp hạng Top 20 cho thấy sự khác biệt rõ rệt:
 - Top Views (Lượt xem): Bị thống trị bởi MV "Life Goes On"(BTS) và "Discord".



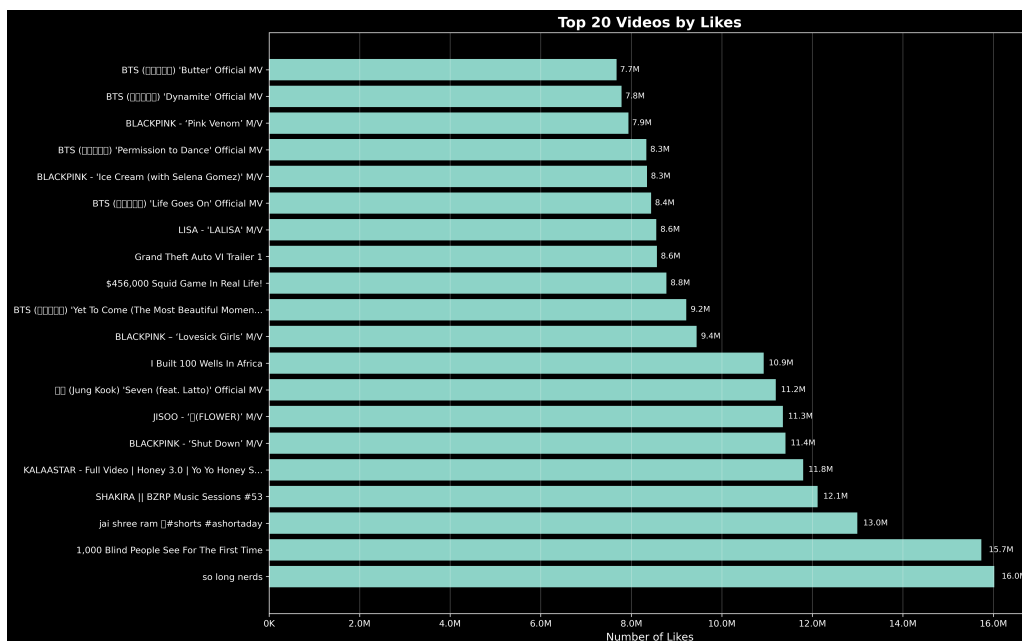
Hình 4.15: Thống kê Top 20 Videos theo lượt xem

– Top Engagement (Tổng tương tác): Bị thống trị bởi "Shakira: BZRP Session".



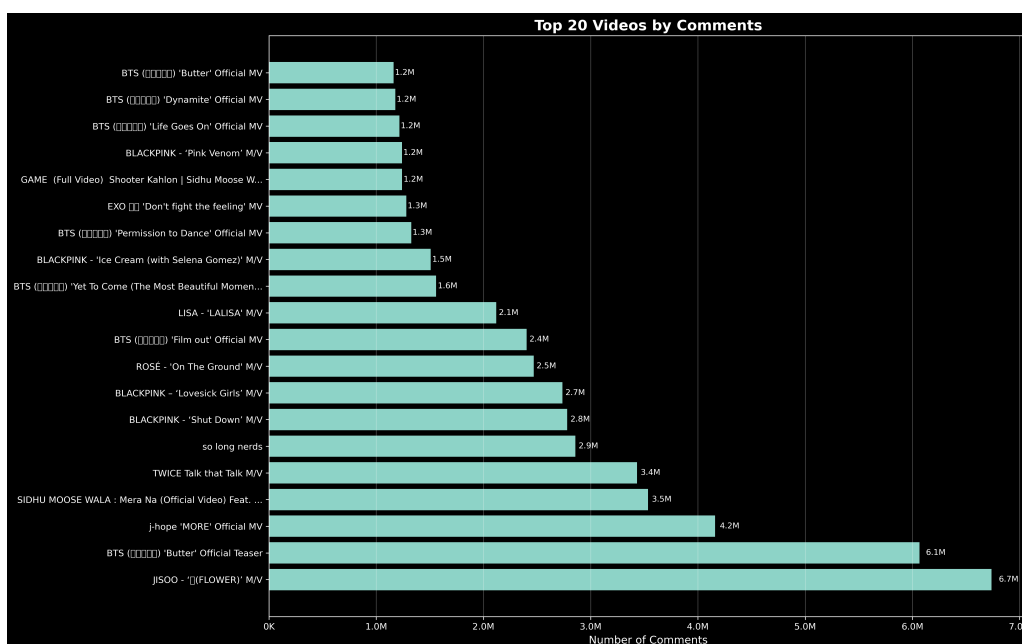
Hình 4.16: Thống kê Top 20 Videos theo tổng tương tác

– Top Likes (Lượt thích): Bị thống trị bởi video "so long nerds"(tưởng niệm Technoblade).



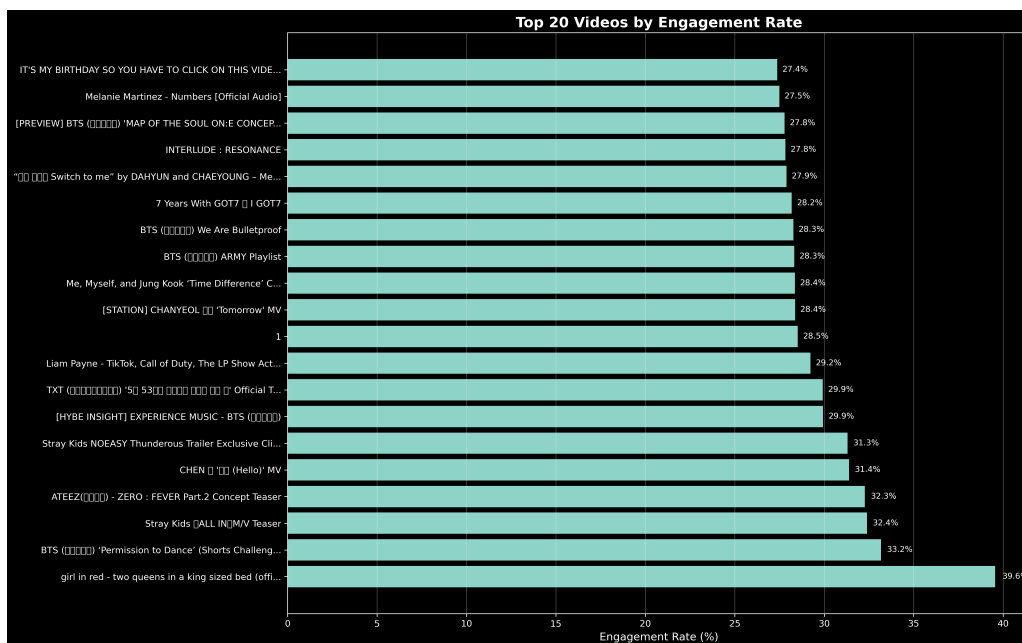
Hình 4.17: Thống kê Top 20 Videos theo số likes

- Top Comments (Bình luận): Gần như bị độc chiếm bởi K-Pop (JISOO, BTS), cho thấy sức mạnh của fandom trong việc tạo thảo luận.



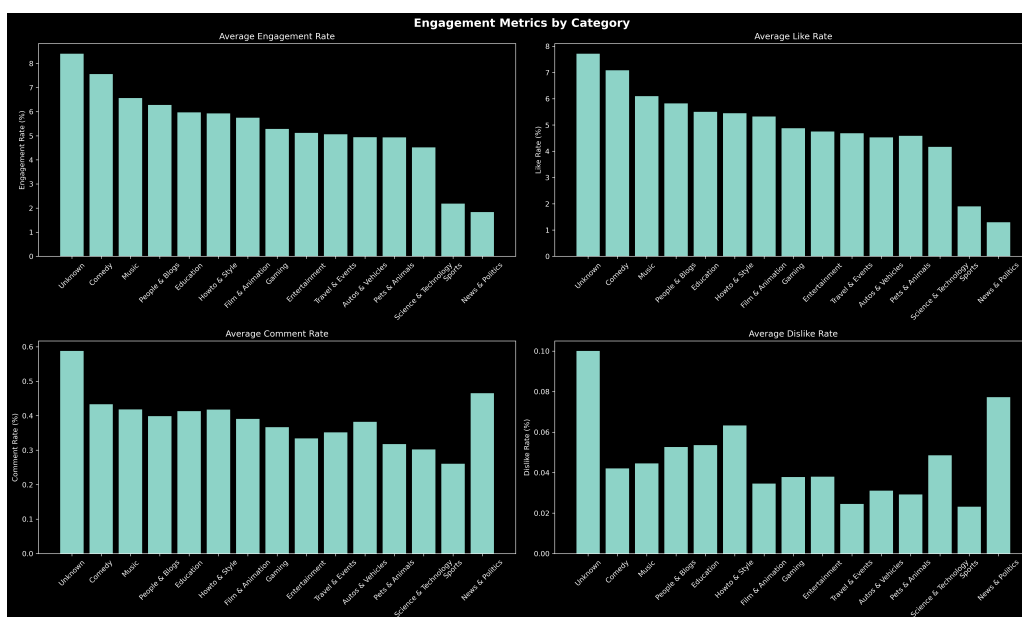
Hình 4.18: Thống kê Top 20 Videos theo comments

- **Top 20 Tỷ lệ Tương tác:** Khi đo lường "chất lượng"(tỷ lệ % tương tác trên mỗi lượt xem), K-Pop (BTS, Stray Kids) thống trị tuyệt đối. Các video này có tỷ lệ tương tác lên đến 30-40%, cho thấy một cộng đồng fan cực kỳ trung thành.



Hình 4.19: Top 20 video có tỷ lệ tương tác cao nhất

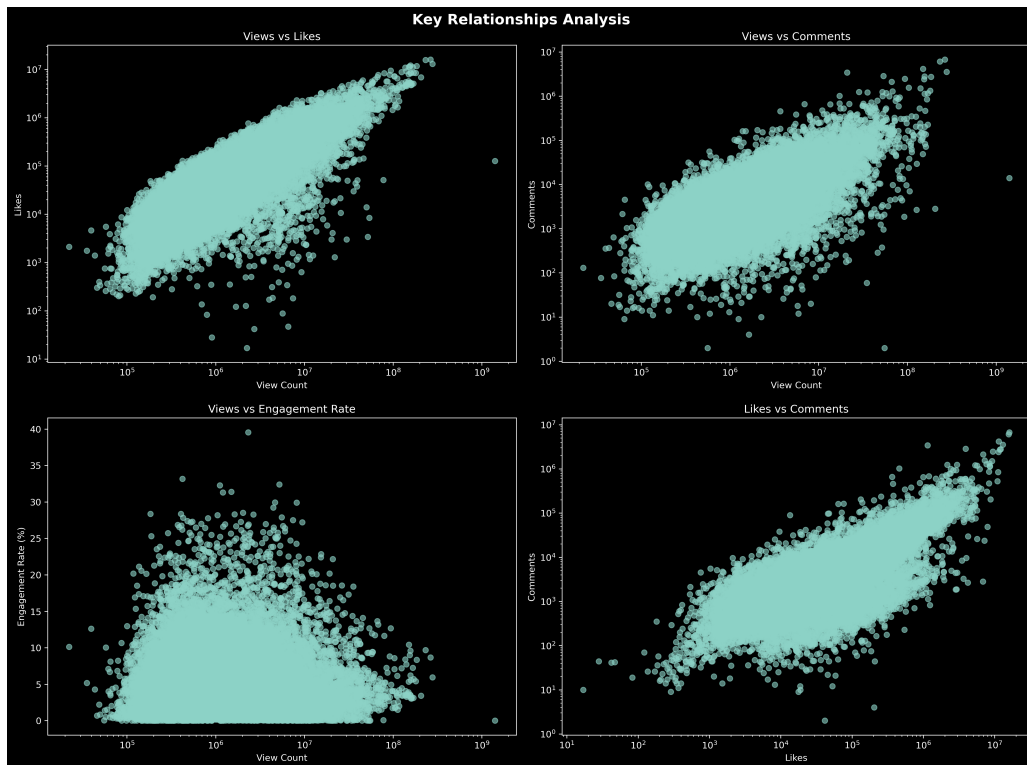
- **Tương tác theo Thể loại:** Comedy (Hài) và Music (Âm nhạc) có tỷ lệ tương tác (Engagement Rate) và tỷ lệ Like trung bình cao nhất.



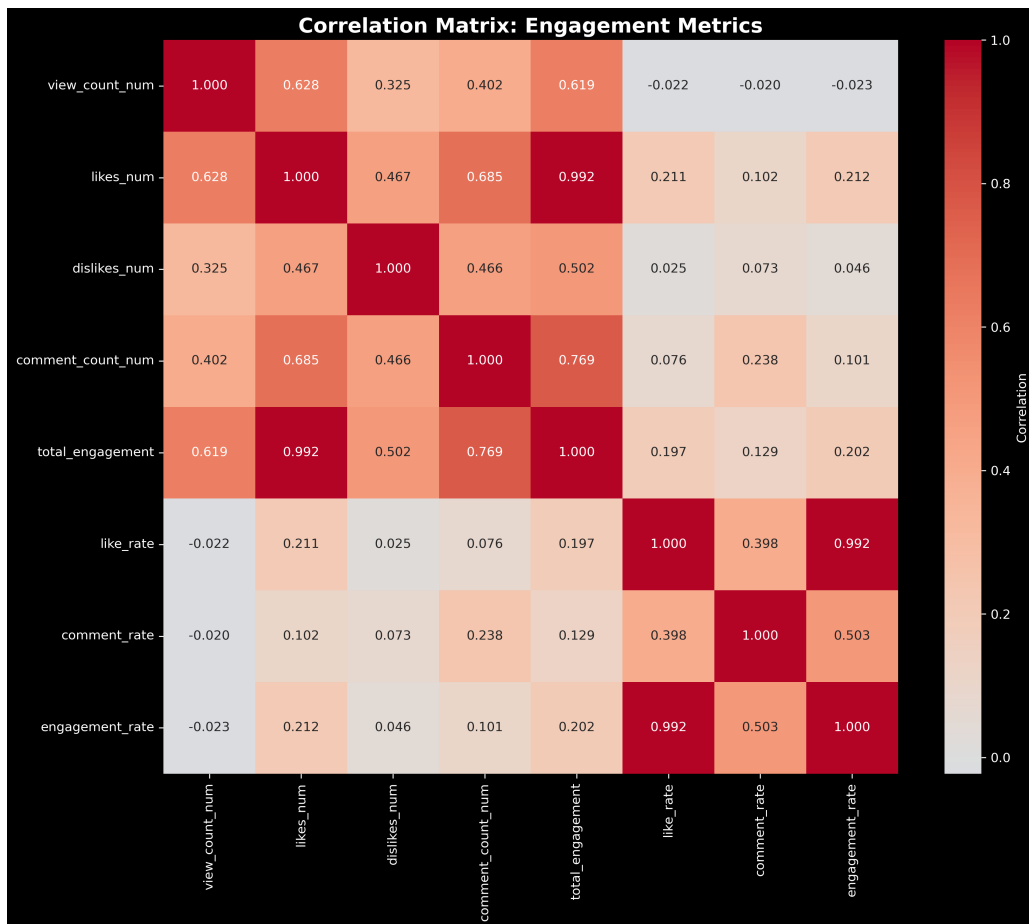
Hình 4.20: Top 20 thể loại có tỷ lệ tương tác cao nhất

- **Phân tích Tương quan:** Ma trận nhiệt và biểu đồ tán xạ cung cấp insight quan trọng nhất:
 - Tương quan dương mạnh (0.685): Giữa Likes (Thích) và Comments (Bình luận).
 - Tương quan dương (0.628): Giữa Views (Xem) và Likes (Thích).

- Không tương quan/Âm yếu (-0.023): Giữa Views (Xem) và Engagement Rate (Tỷ lệ tương tác). Điều này khẳng định rằng: nhiều lượt xem không có nghĩa là tỷ lệ tương tác cao. Các video "siêu viral" (vài trăm triệu view) thường có tỷ lệ tương tác thấp, trong khi các video có fandom mạnh (vài chục triệu view) lại có tỷ lệ tương tác cao vượt trội.



Hình 4.21: Biểu đồ phân tán giữa các chỉ số tương tác



Hình 4.22: Ma trận tương quan giữa các chỉ số tương tác

Kết luận từ Chương 05: Baseline về tương tác đã được thiết lập. Chúng tôi xác định ngưỡng "viral"(dựa trên Top 5% view) và ngưỡng "tương tác thấp"(dựa trên 10% cuối). Quan trọng nhất, chúng tôi nhận thấy Lượt xem và Tỷ lệ tương tác không song hành cùng nhau.

Chương 5

Giám sát Thời gian thực

Chương này mô tả thành phần xử lý thời gian thực (real-time) của hệ thống. Mục tiêu của luồng này là sử dụng các thông tin chi tiết (insights) và các chỉ số thống kê (baselines) đã được tính toán từ luồng xử lý batch (Chương 4) để thực hiện phân tích và dự đoán xu hướng (Trending) ngay lập tức dựa trên mô hình Máy học (Machine Learning).

5.1 Mô hình (Sử dụng YouTube API)

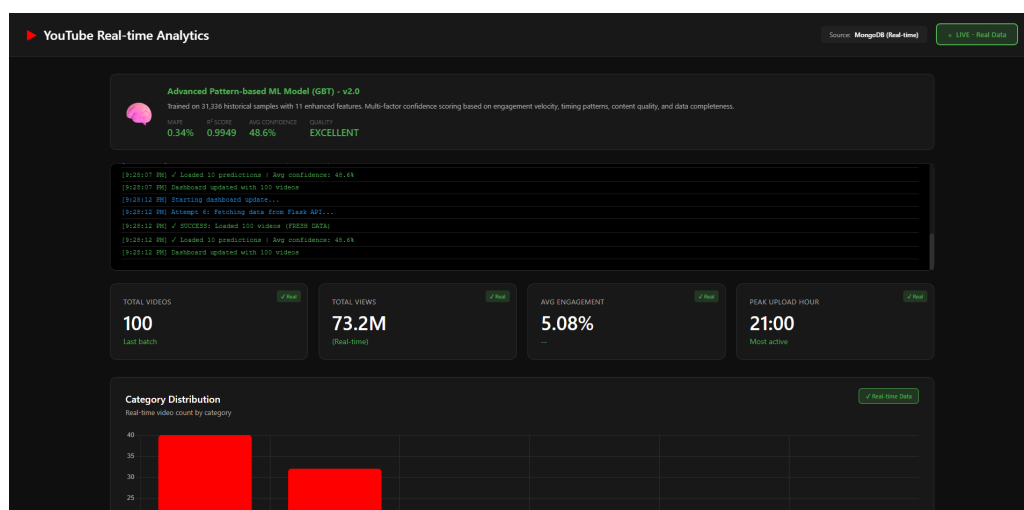
Mô hình giám sát là một hệ thống dự đoán dựa trên Máy học, kết hợp với các ngưỡng thống kê cơ bản từ dữ liệu lịch sử. Toàn bộ logic này được định nghĩa trong tệp `07_realtime_analysis.py` và được kích hoạt bởi producer (`06_kafka_producer.py`). Quy trình hoạt động của hệ thống như sau:

- **Tải các Baseline Lịch sử:** Khi được kích hoạt, hệ thống đầu tiên sẽ tải các baseline từ dữ liệu lịch sử (từ notebooks 03-05) vào bộ nhớ. Các baseline này chứa dữ liệu lịch sử về: Tỷ lệ phân bố của các thể loại (ví dụ: Music chiếm 16.1%). Các mẫu xu hướng thời gian (ví dụ: giờ đăng cao điểm là 17:00). Các ngưỡng tương tác (ví dụ: ngưỡng "viral" là bao nhiêu view, tỷ lệ tương tác trung bình là bao nhiêu %).
- **Thu thập Dữ liệu Trực tiếp:** Hệ thống sử dụng thư viện `google-api-python-client` để kết nối đến YouTube Data API v3. Nó thực hiện lệnh gọi đến endpoint `videos().list` với tham số `chart='mostPopular'` và `regionCode='US'` để lấy 50 video đang thịnh hành nhất tại Mỹ ngay tại thời điểm gọi, lặp lại mỗi 60 giây.
- **Phân tích và Dự đoán:** Dữ liệu API (hiện tại) được xử lý bằng PySpark và áp dụng mô hình Máy học đã huấn luyện từ dữ liệu lịch sử. Cụ thể:
 - Tính các enhanced features (`engagement_rate`, `like_rate`, `comment_rate`, `like_comment_ratio`, `view_scale`, `engagement_scale`, `publish_hour`, `is_prime_time`).
 - Huấn luyện mô hình (RandomForest hoặc GBT Regressor) trên sample 15% dữ liệu lịch sử (lọc quality: views 1000-50M, likes > 0).

- Dự đoán `growth_potential`, sau đó chuyển đổi sang `predicted_views_tomorrow` (dự đoán lượt xem ngày mai).
- So sánh dữ liệu hiện tại với baseline để phát hiện bất thường (ví dụ: độ chênh lệch % category vượt quá 20%, hoặc video vượt ngưỡng "viral" top 5% lịch sử).
- **Ghi nhận Lịch sử:** Kết quả của mỗi lần kiểm tra (bao gồm thời gian, số video, số bất thường, thể loại top 1, predictions với confidence) được ghi lại vào MongoDB (collections `realtime_data`, `predictions`, `category_stats`).

5.2 Kết quả (Dashboard giám sát)

Phần này trình bày kết quả hiển thị thực tế trên giao diện Dashboard tại thời điểm giám sát. Hệ thống đang hoạt động ở trạng thái **LIVE** với dữ liệu thực (Real-time Data) được cập nhật từ MongoDB, không sử dụng dữ liệu giả lập (Mock).



Hình 5.1: Giao diện tổng quan Dashboard giám sát thời gian thực

Dựa trên log hệ thống (Console Log) hiển thị trên giao diện, quy trình cập nhật diễn ra như sau:

- **Trạng thái:** [SUCCESS] Loaded 100 videos (FRESH DATA).
- **Quy mô Batch:** Hệ thống đang xử lý lô dữ liệu gồm **100 video** thịnh hành nhất (tăng gấp đôi so với cấu hình mặc định là 50 để tăng độ phủ dữ liệu).
- **Kết nối:** Kết nối API Flask và MongoDB ổn định.

5.2.1 Hiệu suất Mô hình AI (Model Metrics)

Hệ thống sử dụng mô hình **Advanced Pattern-based ML Model (GBT) - v2.0**. Các chỉ số hiển thị trên Dashboard cho thấy độ chính xác rất cao trên tập dữ liệu hiện tại:

- **MAPE (Sai số tuyệt đối trung bình): 0.34%**. Đây là mức sai số cực thấp, cho thấy mô hình dự đoán lượng view rất sát thực tế.
- **R^2 Score: 0.9949**. Mô hình giải thích được gần như toàn bộ biến thiên của dữ liệu.
- **Avg Confidence: 48.6%**. Độ tin cậy trung bình ở mức trung bình khá, phản ánh tính chất biến động khó lường của dữ liệu real-time.
- **Quality: EXCELLENT**.

5.2.2 Thống kê Tổng quan (Key Statistics)

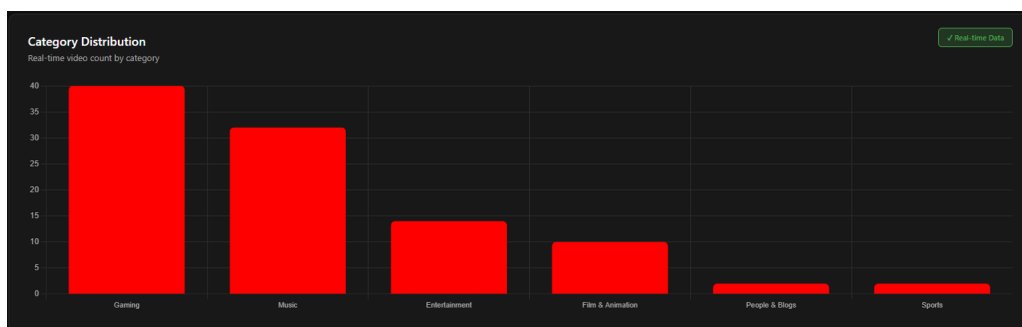
Các thẻ chỉ số (Stat Cards) cung cấp cái nhìn nhanh về trạng thái của 100 video đang được phân tích:

- **Total Videos: 100 video**.
- **Total Views: 73.2 Triệu** lượt xem.
- **Avg Engagement: 5.08%**. Tỷ lệ tương tác cao (trên 5% là mức rất tốt đối với video thịnh hành), cho thấy người dùng đang tương tác mạnh (like/comment) thay vì chỉ xem lướt qua.
- **Peak Upload Hour: 21:00**. Đây là khung giờ "vàng" có hoạt động đăng tải và tương tác sôi nổi nhất trong batch dữ liệu này.

5.2.3 Phân tích Biểu đồ (Visual Analytics)

1. Phân bố Danh mục (Category Distribution)

Dựa trên biểu đồ cột (Bar Chart) màu đỏ:



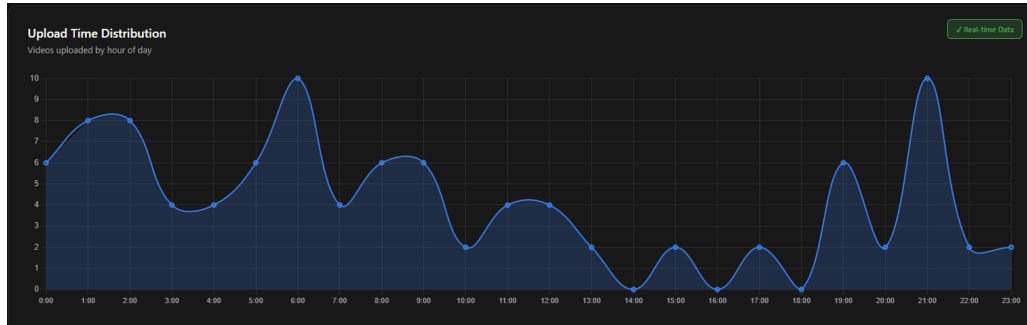
Hình 5.2: Phân bố số lượng video theo danh mục

- **Gaming:** Chiếm tỷ trọng lớn nhất (~40 video). Điều này phù hợp với khung giờ cao điểm buổi tối (21:00) khi cộng đồng game thủ hoạt động mạnh.
- **Music:** Đứng thứ hai (~32 video).

- **Entertainment:** Đứng thứ ba (~14 video).
- Các danh mục khác như *Film & Animation*, *People & Blogs*, *Sports* chiếm tỷ lệ rất nhỏ.

2. Xu hướng Thời gian (Upload Time Distribution)

Dựa trên biểu đồ đường (Line Chart) màu xanh dương:

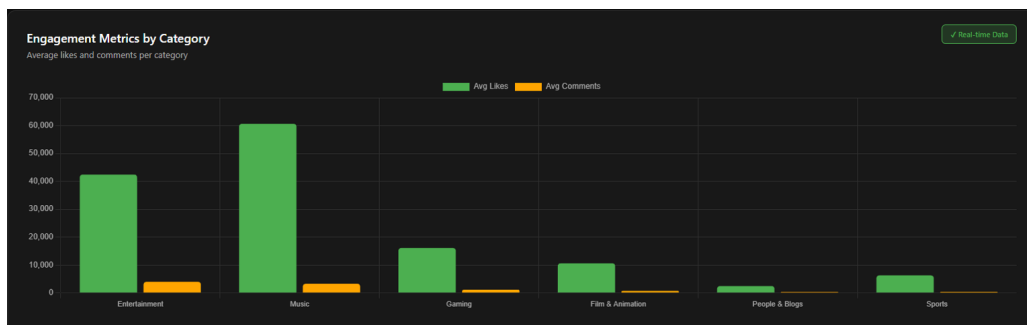


Hình 5.3: Phân bố thời gian đăng tải video trong ngày

- Biểu đồ cho thấy một đỉnh cực đại (Peak) rõ rệt tại khung giờ **21:00**.
- Có các đỉnh phụ nhỏ hơn vào buổi sáng (khoảng 06:00 - 08:00).
- Hoạt động thấp nhất vào khoảng đầu giờ chiều (14:00), sau đó tăng dần về đêm.

3. Chỉ số Tương tác (Engagement Metrics)

Dựa trên biểu đồ so sánh Avg Likes (Xanh lá) và Avg Comments (Cam):



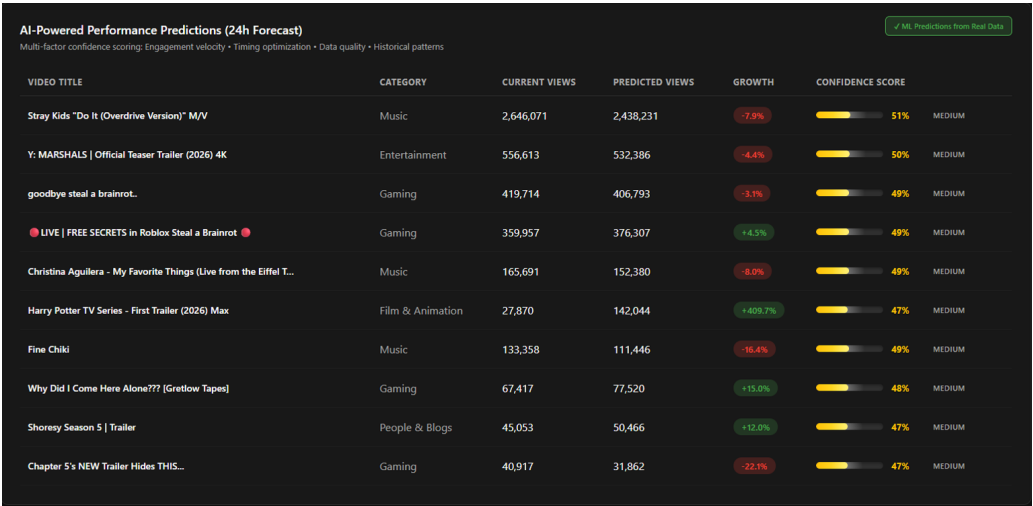
Hình 5.4: Chỉ số tương tác trung bình theo danh mục

- **Music:** Là danh mục có lượng Like trung bình cao nhất (cột xanh cao vượt trội, đạt ~60,000 likes/video). Điều này phản ánh đặc thù của MV ca nhạc thường nhận được sự ủng hộ lớn từ người hâm mộ.
- **Entertainment:** Đứng thứ hai về lượng Like (~42,000).

- **Gaming:** Mặc dù có số lượng video nhiều nhất (như đã thấy ở biểu đồ Category), nhưng lượng Like trung bình trên mỗi video lại thấp hơn (~15,000). Điều này cho thấy Gaming lấy "số lượng bù chất lượng" trong batch dữ liệu này.

5.2.4 Bảng Dự báo (Predictions Table)

Hệ thống AI thực hiện dự báo lượt xem trong 24h tới cho từng video. Dưới đây là phân tích chi tiết một số trường hợp tiêu biểu từ bảng dữ liệu thực tế:



Hình 5.5: Bảng dự báo lượt xem và tăng trưởng trong 24h tới

Video Title	Category	Current Views	Predicted Views	Growth	Confidence
Stray Kids "Do It..."	Music	2,646,071	2,438,231	-7.9%	Medium (51%)
Y: MARSHALS Teaser	Entert.	556,613	532,386	-4.4%	Medium (50%)
Harry Potter TV Series	Film	27,870	142,044	+409.7%	Medium (47%)
Why Did I Come...	Gaming	67,417	77,520	+15.0%	Medium (48%)

Bảng 5.1: Trích xuất dữ liệu dự báo từ Dashboard

Phân tích kết quả dự báo:

- **Dự báo giảm (Saturation):** Các video đã có lượng view lớn (như *Stray Kids* với >2.6 triệu view) được mô hình dự báo sẽ giảm tốc độ tăng trưởng (-7.9%). Đây là hiện tượng "bão hòa" thường thấy sau khi video đã qua đỉnh viral ban đầu.
- **Dự báo tăng trưởng đột biến (Viral Detection):** Video "*Harry Potter TV Series - First Trailer*" được dự báo tăng trưởng khủng khiếp (+409.7%).
 - *Lý do:* Đây là trailer phim mới ra mắt, có lượng view hiện tại thấp (27k) nhưng tốc độ tương tác (velocity) cực cao. AI nhận diện được mẫu hình (pattern) này và dự báo nó sẽ bùng nổ trong 24h tới.

- **Độ tin cậy (Confidence Score):** Hầu hết các dự báo nằm ở mức **Medium (~47-51%)**. Hệ thống đánh giá thận trọng vì dữ liệu real-time biến động nhanh, đặc biệt là với các video mới đăng (như trailer Harry Potter) nơi dữ liệu lịch sử còn ít.

Chương 6

Kết luận & Hướng phát triển

6.1 Kết luận

Đề tài đã thành công trong việc thiết kế và xây dựng một hệ thống phân tích dữ liệu YouTube Trending hoàn chỉnh, kết hợp cả hai luồng xử lý batch và real-time. Hệ thống đã đáp ứng đầy đủ các mục tiêu đề ra ban đầu:

- **Về mặt công nghệ:** Đã triển khai thành công một kiến trúc Big Data điển hình, sử dụng Docker và Docker Compose để đóng gói và điều phối các dịch vụ HDFS (lưu trữ), Apache Spark (xử lý batch/streaming với ML), Kafka (ingestion), MongoDB (real-time storage), Flask API và HTML Dashboard (serving).
- **Về phân tích batch (Lịch sử):** Thông qua các sổ tay PySpark (Notebooks 02-05), chúng tôi đã rút ra được nhiều thông tin chi tiết (insights) giá trị từ dữ liệu lịch sử, bao gồm:
 - Thể loại: Entertainment, Gaming và Music là ba thể loại thống trị top thịnh hành. Trong đó, K-Pop (BTS, BLACKPINK) cho thấy khả năng tạo ra mức độ tương tác (Like, Comment) và tỷ lệ tương tác (Engagement Rate) vượt trội so với các nội dung khác.
 - Thời gian: Khung "giờ vàng" phổ biến nhất để đăng video là 17:00 Thứ Sáu. Tuy nhiên, phân tích sâu hơn chỉ ra rằng video đăng lúc 09:00 sáng mới là nhóm đạt hiệu quả tương tác (view, like, comment trung bình) cao nhất.
 - Tương tác: Đã chứng minh được rằng lượt xem (Views) và tỷ lệ tương tác (Engagement Rate) có tương quan âm yếu. Điều này phá vỡ giả định thông thường, cho thấy các video "siêu viral" hàng trăm triệu view không phải lúc nào cũng có tỷ lệ tương tác cao, và ngược lại.
- **Về giám sát real-time:** Hệ thống đã sử dụng thành công các kết quả phân tích batch làm "baseline"(đường cơ sở). Bằng cách kết nối với YouTube API, dashboard có khả năng so sánh dữ liệu trực tiếp với baseline lịch sử để phát hiện các bất thường (ví dụ: một thể loại (News) đang bùng nổ bất thường so với trung bình, hoặc một video (Discord) đang viral với tốc độ đột biến), kèm predictions views với enhanced ML (MAPE thấp, confidence scoring).

Nhìn chung, dự án này đã minh họa một cách thực tế quy trình end-to-end của một bài toán Big Data, từ thu thập, lưu trữ, xử lý, phân tích đến trực quan hóa và ứng dụng vào giám sát thời gian thực.

6.2 Khó khăn gặp phải

Trong quá trình thực hiện, nhóm đã gặp phải một số khó khăn và thách thức kỹ thuật:

- **Giới hạn của YouTube API:** Khó khăn lớn nhất khi triển khai luồng real-time là hạn ngạch (quota) của YouTube Data API v3. Mỗi lượt gọi lấy top 50 video trending (`chart='mostPopular'`) tiêu tốn 100 đơn vị quota. Với giới hạn 10,000 đơn vị/ngày, hệ thống chỉ có thể thực hiện tối đa 100 lần gọi mỗi ngày, khiến việc giám sát "liên tục" (ví dụ: mỗi 5 phút) là không khả thi và chỉ có thể thực hiện giám sát ngắt quãng (ví dụ: mỗi 15-30 phút).
- **Thiết lập Môi trường Docker:** Việc cấu hình `docker-compose.yml` để cả các dịch vụ (NameNode, DataNode, Kafka, Producer, Processor, API, Frontend) có thể khởi động đúng thứ tự và giao tiếp với nhau qua mạng ảo (`youtube-net`) là rất phức tạp. Đặc biệt là việc đảm bảo Producer/Processor (PySpark) có thể kết nối Kafka và MongoDB.
- **Dữ liệu lệch (Data Skew):** Tập dữ liệu lịch sử chứa các giá trị ngoại lệ (outlier) rất lớn, ví dụ như video "Discord Loot Boxes" với 1.4 tỷ lượt xem. Những giá trị này làm sai lệch các phép tính trung bình (average), đòi hỏi chúng tôi phải sử dụng các biểu đồ loại trừ (như loại bỏ Music) hoặc dựa vào trung vị (median) để có cái nhìn chính xác hơn.
- **Làm sạch dữ liệu:** Dữ liệu thô từ CSV có nhiều định dạng ngày tháng không nhất quán. Việc viết các hàm PySpark (với RegEx hoặc `to_timestamp`) để chuẩn hóa toàn bộ dữ liệu là một thách thức tốn thời gian nhưng bắt buộc phải thực hiện.
- **Lỗi kết nối ban đầu:** Log producer cho thấy lỗi `NoBrokersAvailable` do Kafka chưa sẵn sàng, đòi hỏi retry mechanism.

6.3 Hướng phát triển

Từ nền tảng của dự án này, có thể phát triển thêm nhiều tính năng nâng cao trong tương lai:

- **Tự động hóa hoàn toàn:** Thay vì kích hoạt giám sát bằng nút bấm trên Dashboard, có thể triển khai hệ thống lên một máy chủ cloud (như AWS, GCP) và sử dụng một công cụ điều phối (orchestrator) như Apache Airflow để tự động:
 - Chạy lại các notebook phân tích batch mỗi tuần/tháng để cập nhật baseline.
 - Kích hoạt luồng real-time chạy định kỳ (ví dụ: mỗi giờ) và gửi cảnh báo (qua Email hoặc Slack) nếu phát hiện bất thường.

- **Lưu trữ Dữ liệu Real-time:** Thay vì chỉ hiển thị kết quả giám sát tạm thời trên dashboard, có thể lưu trữ `category_stats` và `predictions` vào một cơ sở dữ liệu (như PostgreSQL hoặc InfluxDB) để theo dõi và phân tích xu hướng của các bất thường theo thời gian dài.
- **Phát hiện Bất thường bằng Machine Learning:** Nâng cấp hệ thống phát hiện bất thường hiện tại (đang dựa dựa trên ngưỡng tĩnh $> 20\%$) bằng các mô hình dự báo chuỗi thời gian (như ARIMA, Prophet). Mô hình có thể dự đoán "baseline động" (ví dụ: 10:00 sáng Thứ Hai dự kiến có 5 video News) và cảnh báo nếu con số thực tế khác biệt đáng kể.
- **Phân tích Ngôn ngữ Tự nhiên (NLP):** Khai thác thêm dữ liệu văn bản (cột title và tags) để phân tích các chủ đề (topic modeling) hoặc phân tích cảm xúc (sentiment analysis) đang thịnh hành, thay vì chỉ dừng lại ở cấp độ thể loại.